

当前进度：已完成《数据结构-第1-5章-绪论至树二叉树.pdf》第一章《绪论》全部正文、典型分析、归纳总结与思维拓展内容整理。

本轮处理：在已完成 1.1.1 与 1.1.2 的基础上，新增并整合 1.2.1、1.2.2、归纳总结、思维拓展；本轮阅读范围为 PDF 第 4 页至第 8 页，未超过 5 页。

下次任务：本 act 只处理第一章；第一章已完成，后续不再追加本章内容。若继续全书任务，应另起 act 从第二章《线性表》开始。

第一章 绪论

本章定位

学习主线

第一章不难，但它决定后面整本数据结构的学习方式：

1. **先理解数据结构是什么**：数据结构不是单纯“存数据”，而是研究数据元素之间的关系，以及这种关系在计算机中如何表示和操作。
2. **再理解算法怎么评价**：同一个问题可以有多种算法，考试重点不是“程序能不能跑”，而是能否分析它的时间复杂度和空间复杂度。
3. **最后建立复杂度思维**：以后学习线性表、树、图、查找、排序时，都会反复比较不同结构和算法在时间、空间上的代价。

本章真正高频的内容主要集中在两块：

- 数据结构的三要素：逻辑结构、存储结构、数据运算。
- 算法效率分析：时间复杂度、空间复杂度，尤其是时间复杂度。

1.1 数据结构的基本概念

1.1.1 基本概念和术语

数据是信息的载体，是描述客观事物属性的数、字符，以及所有能输入到计算机中并被计算机程序识别和处理的符号的集合。对计算机而言，数据就是程序加工处理的原料。

数据元素是数据的基本单位，通常作为一个整体进行考虑和处理。一个数据元素可以由若干个数据项组成，而数据项是构成数据元素的不可分割的最小单位。

例如，一条学生记录可以看作一个数据元素，它内部可能包含学号、姓名、性别、专业等数据项。

数据对象是具有相同性质的数据元素的集合，是数据的一个子集。例如，整数数据对象可以表示为：

$$N = \{0, \pm 1, \pm 2, \dots\}$$

数据类型是一个值的集合以及定义在该集合上的一组操作的总称。这里要注意，数据类型不只是“值的范围”，还包括“能做什么操作”。

常见数据类型可分为：

类型	含义	例子
原子类型	值不可再分的数据类型	整型、字符型、布尔型
结构类型	值可以继续分解为若干分量	数组、结构体、记录
抽象数据类型	一个数学模型及定义在该模型上的一组操作	栈、队列、线性表

抽象数据类型（ADT）的关键在于“抽象”：它重点规定数据对象、数据关系和操作，而不直接关心底层如何存储、如何实现。

数据结构是相互之间存在一种或多种特定关系的数据元素的集合。数据元素不是孤立存在的，它们之间的关系称为结构（Structure）。

数据结构包括三方面内容：

要素	关注点	简单理解
逻辑结构	数据元素之间有什么关系	从人的思维角度看结构
存储结构	这种关系在计算机中如何表示	从机器实现角度看结构
数据运算	可以对数据做哪些操作	从算法处理角度看结构

需要特别区分：

- **算法设计**主要取决于逻辑结构。
- **算法实现**依赖于存储结构。

也就是说，逻辑结构回答“数据之间是什么关系”，存储结构回答“这种关系在内存里怎么放”，算法回答“基于这种结构怎么处理”。

1.1.2 数据结构三要素

数据结构三要素是第一章的核心框架：逻辑结构、存储结构、数据运算。后面每学一种结构，都可以从这三个角度拆开理解。

1. 数据的逻辑结构

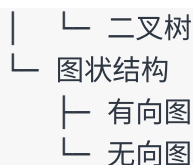
逻辑结构是指数据元素之间的逻辑关系，即从逻辑角度对数据的描述方式。它与数据在计算机中的存储方式无关，是独立于具体系统的。

数据的逻辑结构可分为线性结构和非线性结构：

数据的逻辑结构

```

├─ 线性结构
│   ├── 一般线性表
│   ├── 栈和队列
│   ├── 串
│   └─ 数组
└─ 非线性结构
    ├── 集合
    ├── 树形结构
    │   └─ 一般树
  
```



几类基本逻辑结构可以这样记：

逻辑结构	数据元素之间的关系	典型例子	记忆方式
集合	除“同属一个集合”外，别无其他关系	集合中的若干元素	只属于同一堆
线性结构	一对一关系	线性表、栈、队列	排成一条线
树形结构	一对多关系	树、二叉树	一个父节点对应多个孩子
图状结构 / 网状结构	多对多关系	图、网络	关系最复杂

最容易混淆的是“线性结构”和“顺序存储”：

- 线性结构是逻辑结构，强调元素之间是一对一关系。
- 顺序存储是存储结构，强调元素在内存中物理位置连续。

线性表可以顺序存储，也可以链式存储；所以不能把“线性”直接理解成“内存连续”。

2. 数据的存储结构

存储结构是指数据结构在计算机中的表示形式，也称物理结构。它包括数据元素本身的表示，以及数据元素之间关系的表示。

常见存储结构有四类：

存储结构	核心思想	优点	缺点
顺序存储	逻辑上相邻的元素，物理位置也相邻	可随机存取，空间利用直接	需要连续空间，可能产生外部碎片
链式存储	用指针表示元素之间的逻辑关系	不要求连续空间，插入删除灵活	需要额外指针空间，只能顺序存取
索引存储	额外建立索引表，索引项通常包含关键字和地址	检索速度快	需要额外索引空间，增删时要维护索引
散列存储	根据关键字直接计算存储地址	查找、插入、删除通常很快	可能发生冲突，依赖散列函数设计

理解存储结构时，要抓住一句话：**存储结构是逻辑结构在计算机中的具体实现。**

例如，线性表是一种逻辑结构，它可以用顺序表实现，也可以用链表实现；树是一种逻辑结构，它可以用链式结构实现，也可以用数组下标关系实现。

3. 数据的运算

数据的运算包括两个层面：

层面	说明	关注点
运算的定义	针对逻辑结构，说明运算功能	“要做什么”
运算的实现	针对存储结构，描述具体步骤	“怎么做”

例如，线性表中的“插入”是一种运算定义；但如果用顺序表实现，插入可能需要移动元素；如果用链表实现，插入主要修改指针。逻辑上都是插入，实现代价却不同。

1.2 算法和算法评价

1.2.1 算法的基本概念

算法（Algorithm）是对特定问题求解步骤的描述，是一个有穷的指令序列，其中每条指令表示一个或多个操作。

一个算法必须具备五个重要特性：

特性	含义	易错点
有穷性	算法必须在执行有限步后结束，且每一步都能在有限时间内完成	不能无限循环
确定性	每条指令必须有明确含义，相同输入只能产生相同输出	不能含糊不清
可行性	每一步都能通过已经实现的基本运算执行有限次完成	不能只是理论上想象
输入	可以有零个或多个输入	输入可以没有
输出	至少有一个输出，且输出与输入之间存在特定关系	输出必须有

这里要特别注意两个考点：

- 1. **输入可以为零个，但输出至少有一个。** 例如打印固定内容的程序可以没有输入，但它仍然要产生输出。
- 2. **确定性不是正确性。** 确定性强调指令含义明确、同样输入对应同样输出；正确性强调算法结果能解决问题。

通常，一个“好”的算法还应该满足以下目标：

目标	含义
正确性	能够正确解决所给问题
可读性	结构清晰，便于理解、阅读和交流
健壮性	对非法或异常输入能做出合理处理
高效率与低存储量需求	执行时间少，额外存储空间少

五个基本特性回答“这是不是一个算法”，四个设计目标回答“这个算法好不好”。考试时如果问算法必须满足什么，优先答五个特性；如果问算法设计目标，再答正确性、可读性、健壮性和效率需求。

1.2.2 算法效率的度量

考点追踪：（算法题）分析时空复杂度（2010—2013、2015、2016、2018—2021、2025）

在算法设计中，正确性只是基础，效率才是衡量算法优劣的关键指标。由于实际运行时间会受到硬件、编译器、编程语言、操作系统等因素影响，不能直接作为通用评价标准，所以通常用**渐近复杂度**描述算法随问题规模增长而产生的资源消耗趋势。

复杂度主要包括：

类型	衡量内容	常用记号
时间复杂度	算法执行时间随问题规模增长的趋势	$T(n)$ 、 $O(f(n))$
空间复杂度	算法额外存储空间随问题规模增长的趋势	$S(n)$ 、 $O(g(n))$

1. 时间复杂度

考点追踪：分析算法的时间复杂度（2011—2014、2017、2019、2022、2025）

时间复杂度描述算法执行时间随问题规模 n 增大而变化的趋势，是关于 n 的渐近函数。

实际分析时，通常用语句频度来估计运行时间。语句频度是某条语句的执行次数。如果算法中所有语句频度之和为 $T(n)$ ，它就是问题规模 n 的函数。

当 n 足够大时，低阶项和常数因子对增长趋势影响很小，所以只关注 $T(n)$ 中增长最快的项。

大 O 记号定义为：若存在正常数 C 和 n_0 ，使得对所有 $n \geq n_0$ ，都有

$$0 \leq T(n) \leq Cf(n)$$

则称

$$T(n) = O(f(n))$$

它表示当 n 足够大时， $T(n)$ 的增长速度不超过 $f(n)$ 的常数倍。

例如：

$$T(n) = 3n^2 + 100n + 5 = O(n^2)$$

$$T(n) = 2^n + n^{100} = O(2^n)$$

因为在渐近分析中，最高阶增长项决定主导趋势。

典型分析：顺序查找的最好、最坏、平均复杂度

在数组 $A[0 \dots n - 1]$ 中逆序查找值 k ：

```
int i = n - 1;
while (i >= 0 && A[i] != k)
```

```
i--; // 基本运算
return i;
```

情况	条件	基本运算次数	时间复杂度
最好情况	$A[n-1] = k$	0	$O(1)$
最坏情况	k 不存在，或在最前面才找到	n 级别	$O(n)$
平均情况	所有输入等概率出现时的期望运行时间	n 级别	$O(n)$

通常用**最坏时间复杂度**评价算法效率，因为它给出了运行时间的确定性上界。

复杂度快速计算规则

1. 加法规则：若两段代码顺序执行，总时间复杂度取较高阶项。

$$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

2. 乘法规则：若一段代码嵌套在另一段代码内部，总时间复杂度为二者之积。

$$O(f(n)) \cdot O(g(n)) = O(f(n)g(n))$$

设 $a\{\}$ 、 $b\{\}$ 、 $c\{\}$ 三个语句块的时间复杂度分别为 $O(1)$ 、 $O(n)$ 、 $O(n^2)$ ，则：

① 顺序结构：

```
a {
    b {}
    c {}
}
```

顺序执行时用加法规则：

$$O(1) + O(n) + O(n^2) = O(n^2)$$

② 嵌套结构：

```
a {
    b {
        c {}
    }
}
```

嵌套执行时用乘法规则：

$$O(1) \cdot O(n) \cdot O(n^2) = O(n^3)$$

常见时间复杂度按增长速度从低到高排列为：

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

这里的对数底数通常不影响复杂度阶数，因为不同底数之间只相差常数倍。

2. 空间复杂度

空间复杂度 $S(n)$ 表示算法在运行过程中所需的额外存储空间，是问题规模 n 的函数，通常记为：

$$S(n) = O(g(n))$$

程序执行时占用的空间通常包括：

空间类型	是否重点计入空间复杂度	说明
指令、常数、与 n 无关的变量	通常不作为增长阶重点	固定开销
输入数据	通常不计入额外空间复杂度	输入本身由问题给出
辅助空间	重点关注	临时数组、递归调用栈、指针等

如果算法创建了长度与输入规模 n 同阶的辅助数组，则空间复杂度为 $O(n)$ 。

如果算法只使用常量级额外空间，则称为原地工作，空间复杂度为：

$$O(1)$$

分析空间复杂度时，要重点看“额外使用的辅助空间是否随 n 增长”。递归算法还要注意递归调用栈的空间消耗。

归纳总结

复杂度分析的两类典型情形

本章复杂度分析的核心是时间复杂度。很多题不是单纯背结论，而是要能根据程序结构推导基本运算执行次数。

1. 循环主体中的变量参与循环条件判断

如果循环变量本身参与循环条件，应建立“执行次数 t ”与“问题规模 n ”之间的关系式，解出 $t = f(n)$ ，再判断复杂度。

例 1：

```
int i = 1;
while (i <= n)
    i = i * 2;
```

设基本运算 $i = i * 2$ 执行 t 次，则有：

$$2^t \leq n$$

解得：

$$t \leq \log_2 n$$

所以：

$$T(n) = O(\log_2 n)$$

例 2：

```
int y = 5;
while ((y + 1) * (y + 1) < n)
    y = y + 1;
```

设基本运算 $y = y + 1$ 执行 t 次，则执行后 $y = t + 5$ 。循环条件给出：

$$(t + 5 + 1)(t + 5 + 1) < n$$

即：

$$(t + 6)^2 < n$$

所以：

$$t < \sqrt{n} - 6$$

因此：

$$T(n) = O(\sqrt{n})$$

这类题的关键不是盲目数循环，而是看循环变量如何变化：乘 2 常对应 $\log n$ ，加 1 常对应 n ，平方边界常对应 $\sqrt{n}$ 。

2. 循环主体中的变量与循环条件无关

这类问题常用数学归纳法或直接累计循环执行次数。分析多层循环时，应从内向外逐层计算，忽略单步语句和条件判断语句，只关注主体语句执行次数。

递归程序通常可以建立递推关系式。例如：

$$T(n) = 1 + T(n - 1)$$

不断展开：

$$T(n) = 1 + T(n - 1) = 1 + 1 + T(n - 2) = \cdots = n - 1 + T(1)$$

所以：

$$T(n) = O(n)$$

非递归程序通常直接累加循环次数；递归程序通常写出递推式再求解。

思维拓展

斐波那契数列的递归与非递归复杂度

斐波那契数列定义为：

$$F(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ F(n-1) + F(n-2), & n > 1 \end{cases}$$

要求分别分析递归算法和非递归算法的时间复杂度。

1. 递归算法

```
int Fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return Fib(n - 1) + Fib(n - 2);
}
```

递归算法的时间消耗来自两次递归调用：

$$T(n) = T(n-1) + T(n-2) + O(1)$$

这会形成大量重复计算。例如求 $F(n)$ 时会求 $F(n-1)$ 和 $F(n-2)$ ，而求 $F(n-1)$ 又会再次求 $F(n-2)$ 。重复子问题不断扩大，导致递归树规模呈指数级增长。

因此，朴素递归算法的时间复杂度可记为：

$$O(2^n)$$

更精确地说，它与斐波那契数列增长速度同阶，约为 $\Theta(\varphi^n)$ ，其中 φ 是黄金分割比。但在一般复杂度分析中，写成指数级 $O(2^n)$ 已足够表达其低效本质。

递归深度最大为 n ，所以递归调用栈的空间复杂度为：

$$O(n)$$

2. 非递归算法

```
int Fib(int n) {
    if (n == 0) return 0;
    int a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

非递归算法用循环从小到大依次计算，只保留最近两个状态。循环执行次数与 n 成线性关系，因此时间复杂度为：

$$O(n)$$

它只使用 a 、 b 、 c 等常量个变量，不需要随 n 增长的辅助数组，因此空间复杂度为：

$$O(1)$$

这道思维拓展题的核心结论是：同样求 $F(n)$ ，递归写法直观但会重复计算，非递归写法避免重复计算，时间复杂度从指数级降为线性级。

本章最终速记

内容	必背结论
数据结构三要素	逻辑结构、存储结构、数据运算
逻辑结构分类	集合、线性结构、树形结构、图状结构
存储结构分类	顺序、链式、索引、散列
算法五特性	有穷性、确定性、可行性、输入、输出
好算法目标	正确性、可读性、健壮性、高效率与低存储量需求
时间复杂度	关注基本运算执行次数随 n 增长的趋势
空间复杂度	关注额外辅助空间随 n 增长的趋势
最常用评价	通常使用最坏时间复杂度
复杂度计算	顺序用加法规则，嵌套用乘法规则

本章最重要的能力不是记定义，而是形成“结构 + 实现 + 代价”的思维：先看数据元素之间是什么关系，再看关系如何存储，最后看操作在这种存储方式下要付出多少时间和空间成本。